# Programming environments and debugging

## CSC103 Programming Fundamentals / Lecture 26

**Dr. Muhammad Farhan**

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

# The problem for this lecture

Extract the demonstrated maximum calculation into max(int[] values). Specify empty-input behaviour and test negative-only data.

Document that the method requires a nonempty array.

# Learning objectives

- Use breakpoints and variable inspection
- Distinguish debugging from refactoring
- Refactor while preserving observable behaviour

CLO-3
A modern programming environment
Debugger operations
Refactoring
A repeatable debugging process

# A modern programming environment

- An IDE integrates editing, compilation, navigation and debugging.
- Project settings select a JDK and source level.
- A clean build reduces confusion from stale compiled files.

## Example

```
Before a demo:
Check the active JDK.
Check the working directory.
Run the intended main class.
```

# Debugger operations

- A breakpoint pauses before a selected statement executes.
- Step over executes a call without entering it. Step into enters a call.
- Inspect variables and stack frames to test a hypothesis.

## Example

```
Hypothesis: the loop misses its final element.
Breakpoint: loop condition
Watch: i, values.length, total
```

# Refactoring

- Refactoring improves internal structure while preserving intended observable behaviour.
- Examples include renaming and extracting a method.
- Run existing tests before and after the change.

## Example

```
Before: repeated discount formula
After: one discount(price, rate) method
Same inputs should produce the same outputs.
```

# A repeatable debugging process

- Reproduce the failure with the smallest useful input.
- Predict the faulty state, inspect it and make a focused correction.
- Keep a regression case that demonstrates the original bug.

## Example

```
Failure: max({-4,-2}) returns 0.
Cause: max starts at 0.
Correction: initialise from the first element.
```

# A minimal failing case guides a repair

- The smallest example that violates the contract is easier to reason about.
- For a maximum function initialised to zero, {-2} already proves the defect.
- The repair needs an empty-input policy before using the first element.

## Example

```
Failure: max({-2}) returns 0.
Required result: -2.
A positive-only test hides this defect.
```

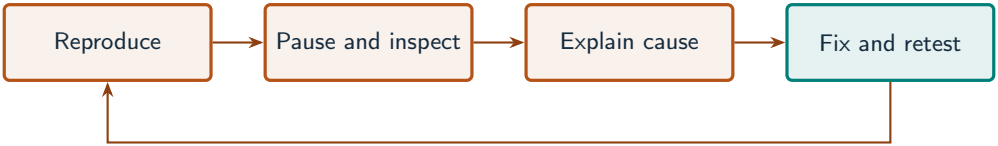
# Extraction should preserve the calculation

- Choose the statements that implement one responsibility.
- Replace their external inputs with parameters and their result with a return value.
- Run the same examples before and after extraction to check behaviour.

## Example

```
Before: loop embedded in main
After: main calls max(values)
The same array should yield the same maximum.
```

# Debugging is a hypothesis-checking cycle




## What to notice

Inspect the first state that differs from your prediction, then rerun the reproducing case.

# Key distinctions

| Observation | Hypothesis | Debugger check |
|---|---|---|
| Negative-only case gives 0 | Bad initial candidate | Inspect maximum before loop |
| Last value ignored | Wrong upper bound | Watch final index |
| Total changes between calls | Shared state retained | Inspect field or accumulator |

These distinctions determine which operation or control structure fits the problem.

# First example: execution

```
int[] values = {-4, -2, -9};
int maximum = values[0];
for (int i = 1; i < values.length; i++) {
  if (values[i] > maximum) maximum = values[i];
}
System.out.println(maximum);
```

# First example: result

-2
Initial maximum: -4. After comparing -2: -2.
Comparing -9 leaves the maximum unchanged.

Refactoring
A repeatable debugging process

# Guided practice

Extract the demonstrated maximum calculation into max(int[] values). Specify empty-input behaviour and test negative-only data.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

# Solution strategy

- Document that the method requires a nonempty array.
- Move the maximum calculation into a method returning int.
- Call it for negative-only data and reject the empty case explicitly.

The next program uses fixed inputs so its result can be reproduced.

# Complete solution (1/2)

```
public class Solution26 {
  public static void main(String[] args) throws Exception {
    System.out.println(max(new int[]{-4, -2, -9}));
    try {
      max(new int[]{});
    } catch (IllegalArgumentException ex) {
      System.out.println("Empty array rejected");
    }
  }
  static int max(int[] values) {
```

# Complete solution (2/2)

```
    if (values == null || values.length == 0) {
      throw new IllegalArgumentException("nonempty array required");
    }
    int maximum = values[0];
    for (int i = 1; i < values.length; i++) {
      if (values[i] > maximum) maximum = values[i];
    }
    return maximum;
  }
}
```

# Execution trace

| Breakpoint | Observed value | Interpretation |
|---|---|---|
| Before loop | maximum = -4 | Valid first candidate |
| After i=1 | maximum = -2 | Larger value replaces it |
| After i=2 | maximum = -2 | -9 cannot replace it |
| Empty input guard | length = 0 | Exception before index access |

Follow the changing state in execution order.

# Solution result

-2
Empty array rejected

Why this result follows
Call it for negative-only data and reject the empty case explicitly.

# A common incorrect approach

```
int maximum = 0;
for (int v : values) maximum = Math.max(maximum, v);
```

## Example

```
This fails for nonempty arrays containing only negative values. Initialise from the first
    element under a nonempty-array contract.
```

# Boundary and failure checks

Check the stated input assumptions.
Record one debugging session: failing input, hypothesis, observed variable values, correction and regression test.

Reject an empty array with IllegalArgumentException. Start from values[0], traverse from index 1 and return maximum. {-4,-2,-9} must give -2.

# Check your understanding

Does rename refactoring intentionally change output?
What is a regression test?

Write a prediction and explain the rule that supports it.

# Answer and explanation

No. It changes a name while preserving behaviour. A regression test checks that a previously corrected failure does not return.

This fails for nonempty arrays containing only negative values. Initialise from the first element under a nonempty-array contract.

# Compare cases and predict outcomes

| Observation | Hypothesis | Useful inspection |
|---|---|---|
| Last element omitted | Loop stops too early | Index and length at condition |
| First item skipped | Counter starts at 1 | Initial counter value |
| Total persists | Accumulator not reset | Value before the next run |

## Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

# Apply the idea

## Independent practice

A sum loop uses i < values.length - 1 for {2,4,6}. Predict the wrong result, identify the missing visit, and correct the condition.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

# Solution and reasoning

```
int sum = 0;
for (int i = 0; i < values.length; i++) {
  sum += values[i];
}
```

- The faulty condition visits indexes 0 and 1 only, giving 6.
- Index 2, containing 6, is omitted.
- The corrected loop visits all three elements and gives 12.

# Debugging an incorrect array average

- The source debugger exercise starts its sum loop at 1 and omits the first element.
- It also divides ints before returning double, so a fractional mean can be lost.
- Use a fractional expected result to detect both defects instead of only testing an integer-valued mean.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

# Worked problem: Debugging an incorrect array average

## Task

Correct the sum and average for {1,2,3,4}. Inspect i, sum and the final divisor with a breakpoint in the loop.

Input: Values are fixed in the program for a reproducible demonstration.

## Before running

Predict the output and identify the variables that change.

# Complete ISB example (1/1)

```
public class IsbLecture26 {
  public static void main(String[] args) throws Exception {
    int[] values = {1, 2, 3, 4};
    if (values.length == 0) throw new IllegalArgumentException("empty");
    int sum = 0;
    for (int i = 0; i < values.length; i++) sum += values[i];
    double average = (double) sum / values.length;
    System.out.println(sum);
    System.out.println(average);
  }

}
```

# Worked trace and expected result

| Implementation | Sum | Returned average |
|---|---|---|
| Starts at 1; integer division | 9 | 2.0 |
| Index fixed only | 10 | 2.0 |
| Index and division fixed | 10 | 2.5 |

## Expected console output

```
10
2.5
```

# Extend the ISB example

## Practice

Why might {1,2,3,4,5} hide the integer-division defect after the loop is fixed?

## Explain your reasoning

Its exact mean is 3.0, so integer division happens to produce the same numeric answer.

# Lecture summary

- breakpoints and variable inspection
- debugging from refactoring
- Refactor while preserving observable behaviour

- Document that the method requires a nonempty array.
- Move the maximum calculation into a method returning int.
- Call it for negative-only data and reject the empty case explicitly.

# After-class practice

Record one debugging session: failing input, hypothesis, observed variable values, correction and regression test.

Syllabus reading
Reference material
Next lecture: Library components and APIs